P R É - P R O J E T O

Simulador de Oficina Mecânica

Projeto em C++ com Programação Orientada a Objetos

1. Identificação

Título do projeto: Simulador de Oficina Mecânica

Integrantes:

Nº	Nome do integrante	Matrícula
1	João Pedro Vidal Alves	2026953478

2. Descrição do problema

Oficinas mecânicas pequenas costumam controlar clientes, veículos, orçamentos e peças em cadernos ou planilhas soltas. Isso causa erros: orçamentos com valores diferentes do cobrado, peças que faltam na hora do serviço e nenhuma noção clara de quanto a oficina gastou e ganhou.

O Simulador de Oficina Mecânica é um programa de terminal, escrito em C++, que reúne esse controle em um só lugar. O usuário (atendente ou mecânico) cadastra clientes e seus veículos, monta orçamentos com serviços e peças, compra peças de fornecedores para o estoque e acompanha a ordem de serviço até a conclusão, com o dinheiro que entrou e saiu registrado no caixa.

O sistema simula o dia a dia de uma oficina e serve para praticar os conceitos de orientação a objetos: classes, encapsulamento, herança e polimorfismo.

3. Funcionalidades

Em linguagem simples, o sistema saberá:

Clientes e veículos

- Cadastrar, listar, buscar, editar e remover clientes.
- Cadastrar veículos (carro, moto ou caminhonete) e ligá-los ao dono.

Estoque e compras

- Cadastrar peças com código, nome, preço de custo, preço de venda e quantidade.
- Registrar compras de peças, aumentando o estoque e registrando o gasto.
- Listar o estoque e avisar quando uma peça estiver acabando.

Orçamentos

- Criar um orçamento para um veículo, escolhendo o mecânico responsável.
- Adicionar serviços (horas de mão de obra) e peças ao orçamento.
- Calcular o valor total e conferir se as peças estão disponíveis no estoque.

- Aprovar ou recusar o orçamento.

Ordens de serviço e caixa

- Transformar um orçamento aprovado em ordem de serviço, iniciar e concluir.
- Ao concluir, dar baixa nas peças usadas e registrar o pagamento no caixa.
- Mostrar o saldo do caixa e um relatório de receitas e despesas.

Dados

- Salvar todas as informações em arquivos ou banco de daods ao sair e carregá-las ao abrir o programa.

4. Diagrama de blocos

O usuário interage apenas com a interface de console. O menu repassa cada pedido à classe Oficina, que coordena os módulos (cadastros, estoque e compras, orçamentos, ordens de serviço e caixa). Os módulos se comunicam entre si nos pontos indicados pelas setas tracejadas, e a camada de persistência lê e grava os arquivos de dados.

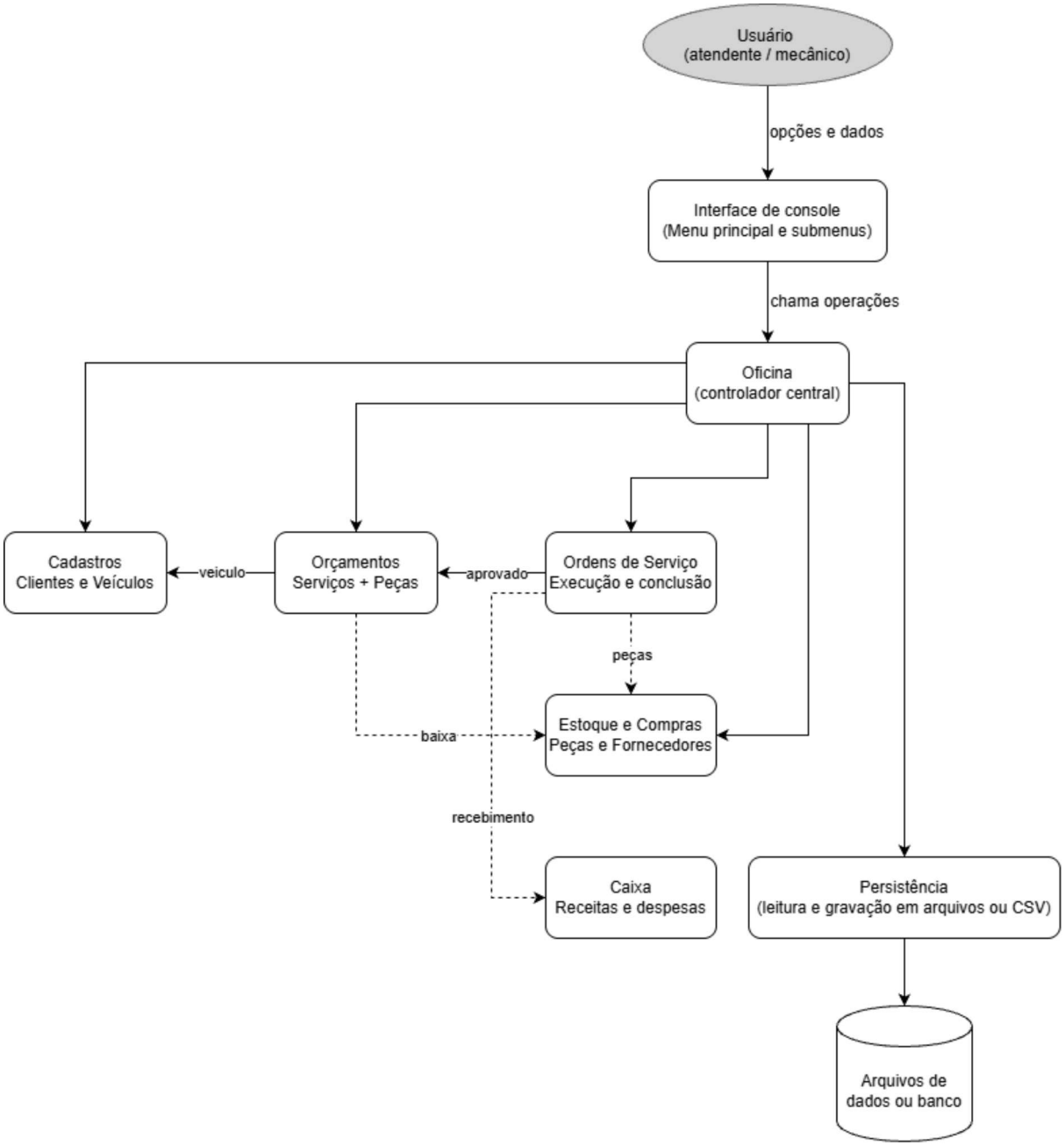


Figura 1 – Diagrama de blocos do sistema

5. Diagrama de classes

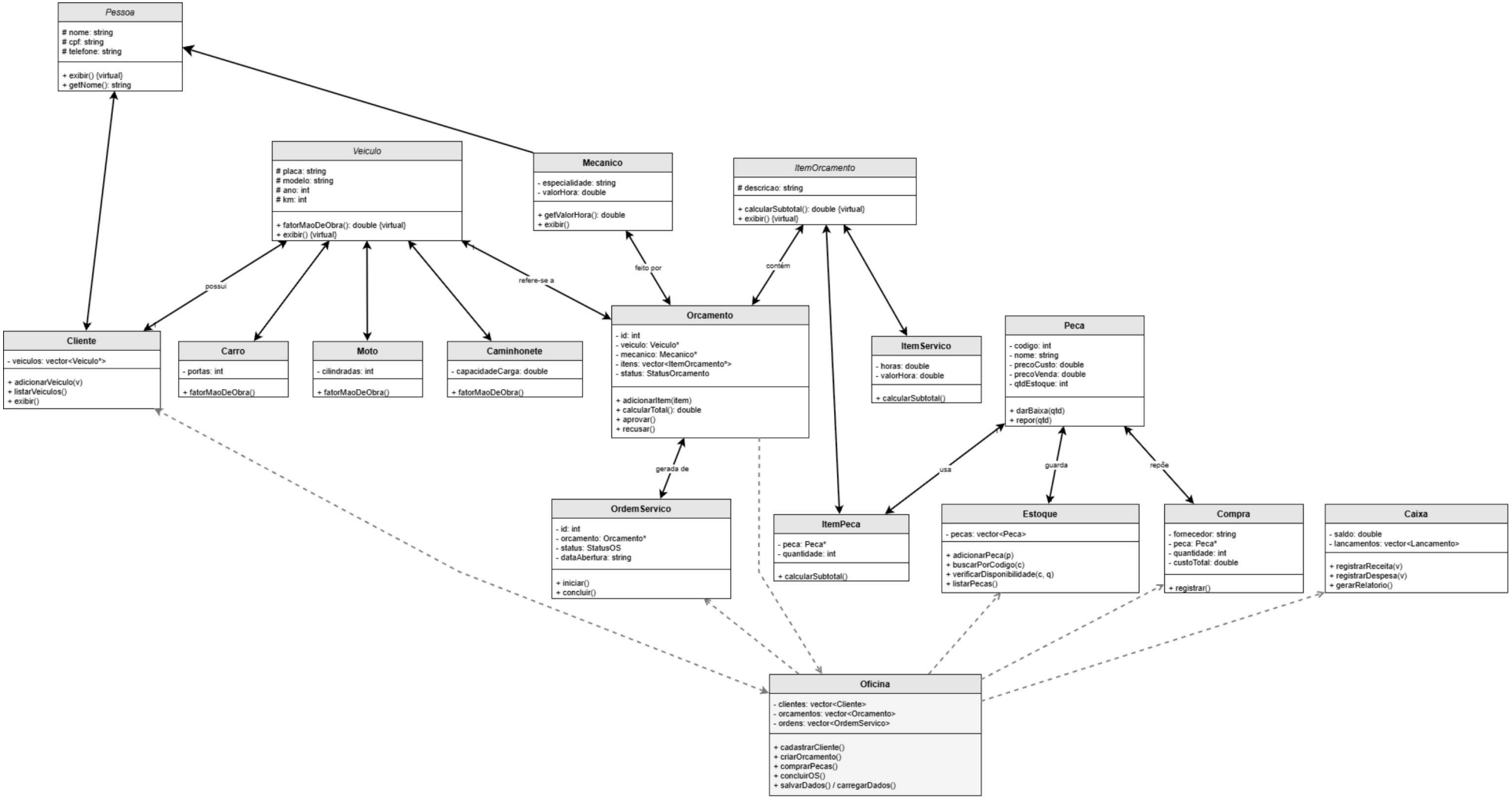


Figura 2 – Diagrama de classes (nomes em itálico = classes abstratas; seta com triângulo vazado = herança; losango = composição/agregação; tracejado = uso pela classe Oficina)

5.1 Heranças previstas

Classe base (abstrata)	Classes derivadas	O que muda em cada uma
Pessoa	Cliente, Mecanico	Cliente guarda a lista de veículos; Mecanico guarda especialidade e valor da hora.
Veiculo	Carro, Moto, Caminhonete	Cada tipo tem atributos próprios e um fator de mão de obra diferente.
ItemOrcamento	ItemPeca, ItemServico	Um item é uma peça (preço × quantidade) ou um serviço (horas × valor da hora).

5.2 Onde o polimorfismo aparece

- **Orçamento:** a classe Orcamento guarda um vector<ItemOrcamento*> e, em calcularTotal(), chama calcularSubtotal() de cada item. O mesmo código calcula peças e serviços, e cada classe filha aplica sua própria conta.
- **Veículos:** fatorMaoDeObra() é virtual em Veiculo. Ao orçar um serviço, o sistema usa o fator do tipo real do veículo (moto, carro ou caminhonete) sem precisar de if/switch por tipo.
- **Listagens:** exibir() é virtual em Pessoa, Veiculo e ItemOrcamento. Uma lista de ponteiros para a classe base pode ser impressa com um único laço.

5.3 Principais relações

- Um Cliente possui zero ou mais Veículos.
- Um Orçamento se refere a um Veículo, é feito por um Mecânico e contém um ou mais itens.
- Um ItemPeca usa uma Peça do Estoque; uma Compra repõe uma Peça.
- Uma OrdemServico nasce de um Orçamento aprovado.
- A classe Oficina reúne clientes, estoque, orçamentos, ordens, compras e caixa, e é o ponto de entrada do menu.

6. Divisão de tarefas

Integrante	Responsabilidades	Classes principais
João Pedro Vidal Alves	Cadastro de clientes e veículos; hierarquias de Pessoa e Veiculo; validação dos dados de entrada.	Pessoa, Cliente, Mecanico, Veiculo, Carro, Moto, Caminhonete
	Estoque de peças, compras de fornecedores e controle do caixa.	Peca, Estoque, Compra, Caixa
	Orçamentos, itens (polimorfismo) e ordens de serviço.	ItemOrcamento, ItemPeca, ItemServico, Orcamento, OrdemServico

Integrante	Responsabilidades	Classes principais
	Classe Oficina, menus de console, gravação e leitura de arquivos, configuração do build e integração das partes.	Oficina, Menu, Persistencia
	Escrever testes das próprias classes, revisar o código dos colegas, manter o README e preparar a apresentação.	–

7. Ferramentas

Ferramenta	Uso no projeto
GitHub	Repositório do código, versionamento com branches por funcionalidade, pull requests com revisão e issues para acompanhar tarefas.
VS	Editor de código, com a extensão C/C++ e depurador integrado.
g++ e CMake	Compilação do projeto em C++17 e organização dos arquivos em include/ e src/.
draw.io	Criação e atualização dos diagramas de blocos e de classes.